ProDock: Project Rundown for Review 2

A plain-English guide to what we built, why, how it works, what the results mean, and how to answer the questions you're likely to get.

1. The project in one paragraph

Docker containers need CPU and memory limits. Set them too high and you waste the machine. Set them too low and the app slows down (CPU) or gets killed (memory). Most people either pick a fixed number and hope, or use a rule like "if CPU is above 80%, give it more." Both react late. Our literature review found that nearly all the smart, AI-based fixes are built for Kubernetes, not for plain Docker running on its own. ProDock fills that gap. It watches each container, predicts how much it will need over the next 3 minutes, and resizes the container with `docker update` before the demand arrives.

2. Which paper we're building on

Our review doesn't name one main paper, so here's the one that fits best, and why. Please confirm this with the team before the review.

Base paper: ConLAR (Chen, Shen and Chen, "ConLAR: Learning to Allocate Resources to Docker Containers under Time-Varying Workloads," IEEE QRS 2021, reference [22] in our review).

It's the right anchor because it's the closest match to the gap we wrote about in Section 5.2. It works directly on Docker containers, not Kubernetes pods. It uses an Observe, Predict, Allocate, Execute loop with an LSTM, which is exactly the skeleton of our system.

Two other papers from our review shaped specific parts:

- **ExtremoNet** ([2], Wang and Wang, Applied Sciences 2024) uses an Isolation Forest to spot extreme CPU values and feeds that information into the prediction model as a feature. We borrowed that idea for our Stage 1.
- **DeepScaling** ([38], Wang et al., IEEE/ACM Transactions on Networking 2024) is about avoiding "thrashing," where an autoscaler flips back and forth. That motivated our scale-down rules.

So the honest one-liner is: **ConLAR's loop, plus ExtremoNet's burst signal, plus anti-thrashing rules, applied to standalone Docker.**

3. Things to fix in the paper before tomorrow

I went through the whole review. It's in good shape overall, but these would get picked on:

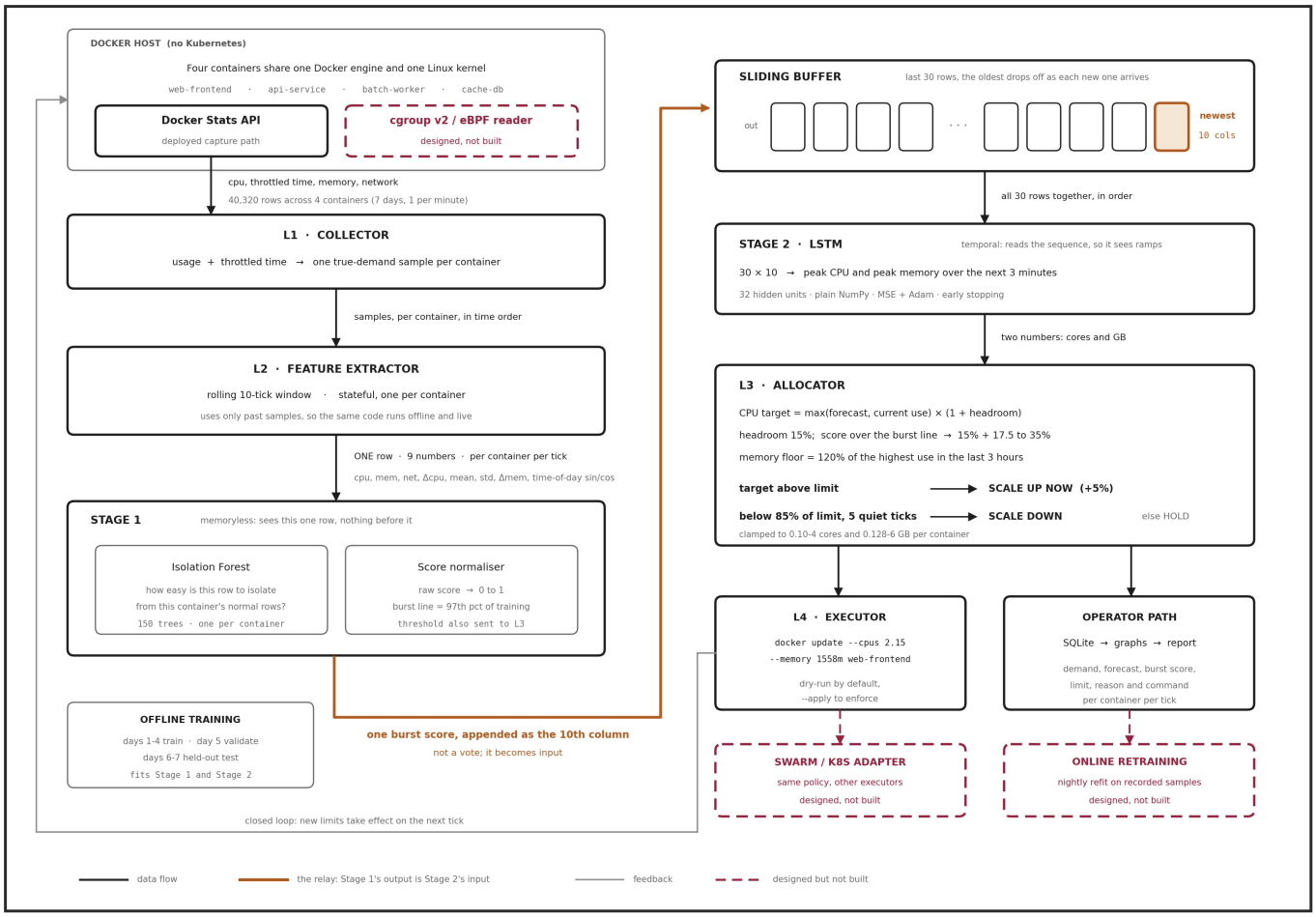
1. **"Various authors" on references [1] to [5] and [8].** Examiners notice this straight away. Two I've confirmed:
   - [1] is John, A., Kawash, J. and Alhajj, R., and the real title ends "using **artificial intelligence** techniques" (not "AI techniques"). Computing, vol. 107, article 150, 2025.
   - [2] is Wang, C. and Wang, Z., Applied Sciences, vol. 14, no. 7, 2911, 2024.
   - [3], [4], [5] and [8] still need real author names.
2. **The ExtremoNet summary (2.2) may be wrong.** Our review says it combines an Isolation Forest with a TCN. The paper's own abstract describes Isolation Forest scores added as features plus classification sub-models. Open the paper and check whether a TCN is actually in there before you say it out loud.
3. **ConLAR's title** is "Time-Varying Workload**s**" (plural) on IEEE Xplore. Our reference and heading say "Workload."
4. **Section 2.7 says the model "likely" combines a metaheuristic with a learning component.** "Likely" reads as "we didn't read it." Either confirm what it does or drop that sentence.
5. **Section 4.2 mentions "Section B" and "Section C."** Those are leftovers. They should say Section 3 and Section 4.
6. **Reference numbering jumps** from [7] to [22] in Section 2. IEEE style numbers references in the order they first appear. Either renumber or be ready to explain the list was extended later.
7. **Reference [5]** has arXiv ID 2311.07818, which means it was first posted in November 2023, but we list 2024. Check which version you mean.
8. **Title page** still says "Member 1 · Member 2 · Member 3." Put your names and the course code in.

9. **Review 2 needs a “Proposed System” section.** Right now the paper stops at the gap. Add a Section 7 with the architecture diagram, the method, the experiment setup and the results table from this rundown.
10. **Section 5.4 (patent note)** says to keep novel implementation details confidential. If a patent is still on the table, check with your guide before sharing the code outside the review.
11. Minor: the 34% saving in [20] comes from a lower-tier journal. Fine to cite, just don’t lean on it as strong evidence.

4. How ProDock works, layer by layer

Think of it like a restaurant manager deciding how many staff to roster. A bad manager rosters the same number every day (static). A slightly better one calls in extra staff once the queue is already out the door (reactive). A good one looks at the last half hour, notices a bus full of tourists just pulled up, and calls people in before they walk in (predictive). ProDock is the good manager.

The architecture diagram below follows this exact flow.



Docker host. Four containers run on one machine with no Kubernetes: a website (`web-frontend`), an API (`api-service`), a scheduled job (`batch-worker`) and a cache/database (`cache-db`). Each behaves differently on purpose.

L1 Collector. Every tick, it reads `docker stats` for each container: CPU used, memory used, network traffic, and throttled time. Throttled time matters. If a container is capped at 1 core but wants 1.5, Docker only reports 1 core used. The throttled time tells us about the missing 0.5. Adding them gives the container's true demand.

L2 Feature extractor. Turns each sample into one row of 9 numbers: CPU, memory, network, how much CPU changed since last tick, the average and wobble (standard deviation) of CPU over the last 10 ticks, how much memory changed, and the time of day written as two numbers (sine and cosine, so 23:59 and 00:00 look close together). It only ever uses past data. That's what lets the exact same code run in training and live.

Stage 1: Burst detector (Isolation Forest). Looks at one row at a time and asks "how weird is this row for this container?" An Isolation Forest builds lots of random trees that keep splitting the data. Normal rows take many splits to separate. Weird rows, like the first minute of a traffic spike, get separated in just a few. Few splits means a high score. We scale the score to 0 to 1 and set a "burst line" at the 97th percentile of training scores.

The relay (orange arrow). The burst score isn't used as a vote. It's added to the row as a 10th column, so the LSTM can learn what a weird row usually leads to. This is the ExtremoNet idea.

Sliding buffer. Keeps the last 30 rows (30 minutes). Every new row pushes the oldest one out.

Stage 2: LSTM. Reads all 30 rows in order and predicts the peak CPU and peak memory over the next 3 minutes. We predict the peak, not the average, because a limit has to cover the highest point. An LSTM is a neural network built for sequences. It has a small memory it updates as it reads each row, so it can pick up patterns like "CPU has been climbing for

4 minutes.” Ours has 32 hidden units and is written in plain NumPy with hand-coded backpropagation, which the test file checks against a numerical gradient.

L3 Allocator. Turns predictions into limits:

- CPU target = the bigger of (forecast, current use) × (1 + headroom).
- Headroom is 15% normally. If the burst score is over the line, it goes up by another 17.5% to 35%, depending on how far over.
- Memory never drops below 120% of the highest use in the last 3 hours. Memory is different from CPU. Too little CPU makes a container slow. Too little memory gets it killed (an “OOM kill”). So we’re deliberately cautious here, similar to how Kubernetes’ Vertical Pod Autoscaler works.
- Growing happens immediately, with 5% extra so tiny creeps don’t trigger another change.
- Shrinking only happens if the new target is under 85% of the current limit and nothing has changed for 5 ticks. This stops thrashing.
- Every container is capped between 0.1 and 4 cores and 0.128 and 6 GB.

L4 Executor. Runs `docker update --cpus X --memory Y container`. This changes the container’s cgroup limits on the fly with no restart, which is why frequent small changes are cheap. It’s in dry-run mode by default and only really changes anything with `--apply`.

Operator path. Every decision is written to SQLite: demand, forecast, burst score, limits, the reason (“up”, “burst-up”, “down”, “hold”) and the exact command. That’s our audit trail and where the graphs come from.

Closed loop. The new limits take effect on the next tick, and the collector sees the result. The loop keeps running.

Designed but not built (red dashed boxes). A direct cgroup/eBPF reader to replace `docker stats`, nightly retraining on recorded data, and adapters for Docker Swarm or Kubernetes. Say clearly that these are future work.

5. How we tested it

We don’t have a real production cluster, so we built a realistic simulated host: 4 containers, 7 days, one reading per minute (40,320 rows).

- `web-frontend` peaks in the evening and gets random flash crowds.
- `api-service` peaks at lunch, is quieter on weekends, and has frequent small spikes.
- `batch-worker` runs a heavy job every 2 hours and is nearly idle otherwise.
- `cache-db` has steady CPU while memory slowly climbs and drops back every night.

Days 1 to 4 train the models. Day 5 is validation (used for early stopping). Days 6 and 7 are the test set, which the models never see during training.

We compare four policies on the same test days:

Policy	What it does
Static	One fixed limit (95th percentile of training use × 1.2). What most people do.
Reactive	Over 80% used? Multiply limit by 1.5. Under 30%? Multiply by 0.75. Like threshold autoscalers.
Predictive (ConLAR-style)	LSTM forecast + 15% headroom, applied every tick. Our base paper’s approach.
ProDock (ours)	Everything in Section 4.

A minute counts as an **SLO violation** if the container wanted more than 2% above its CPU limit.

6. Results and what to say about each graph

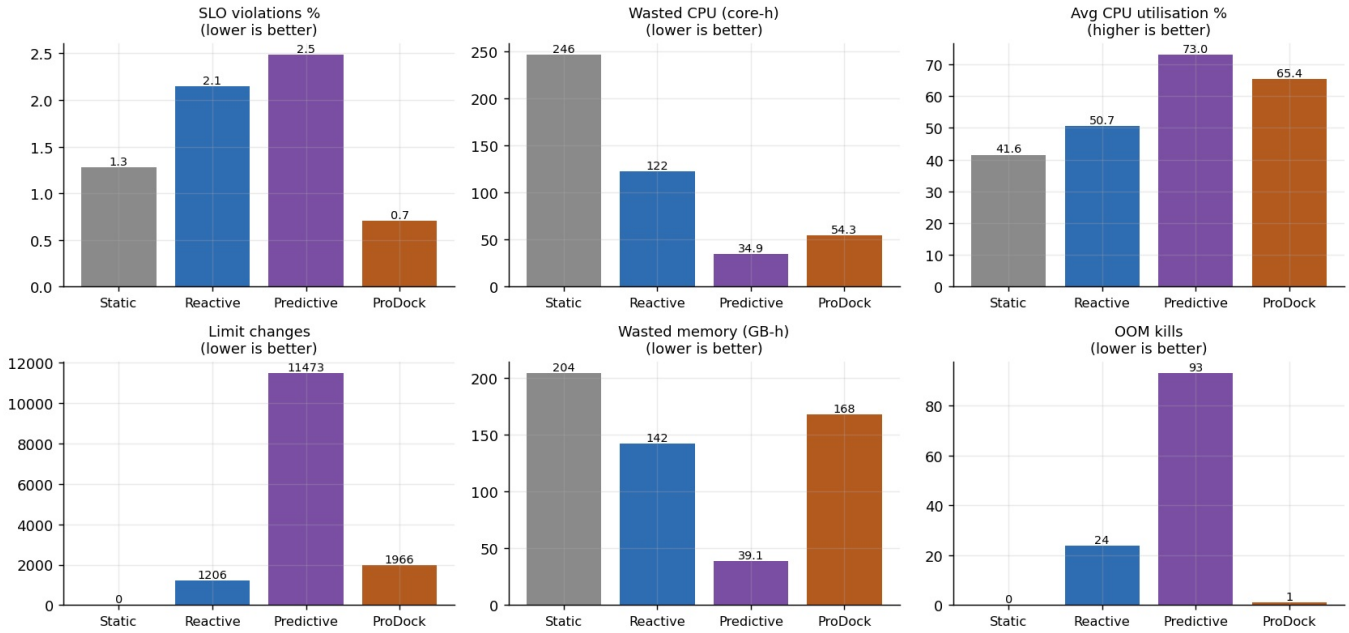
Main results table (test days, all 4 containers)

Policy	SLO violations	Wasted CPU (core-hours)	Avg CPU use	OOM kills	Limit changes
Static	1.28%	246.5	41.6%	0	0
Reactive	2.14%	122.8	50.7%	24	1,206
Predictive (ConLAR-style)	2.48%	34.9	73.0%	93	11,473
ProDock	**0.70%**	54.3	65.4%	**1**	1,966

The headline: ProDock had the fewest SLO violations of all four, while wasting 78% less CPU than static limits and 56% less than reactive rules. Compared with the ConLAR-style predictor, it made 5.8× fewer limit changes and had 1 OOM kill instead of 93.

The honest trade-off: the ConLAR-style predictor wastes less CPU (34.9 vs 54.3) and far less memory. It gets there by cutting things fine, which is why it has 3.5× our violations and 93 OOM kills. ProDock spends a bit more to stay safe.

Fig 7. Head-to-head on the 2 test days (all 4 containers)



Forecast accuracy (peak CPU over the next 3 minutes)

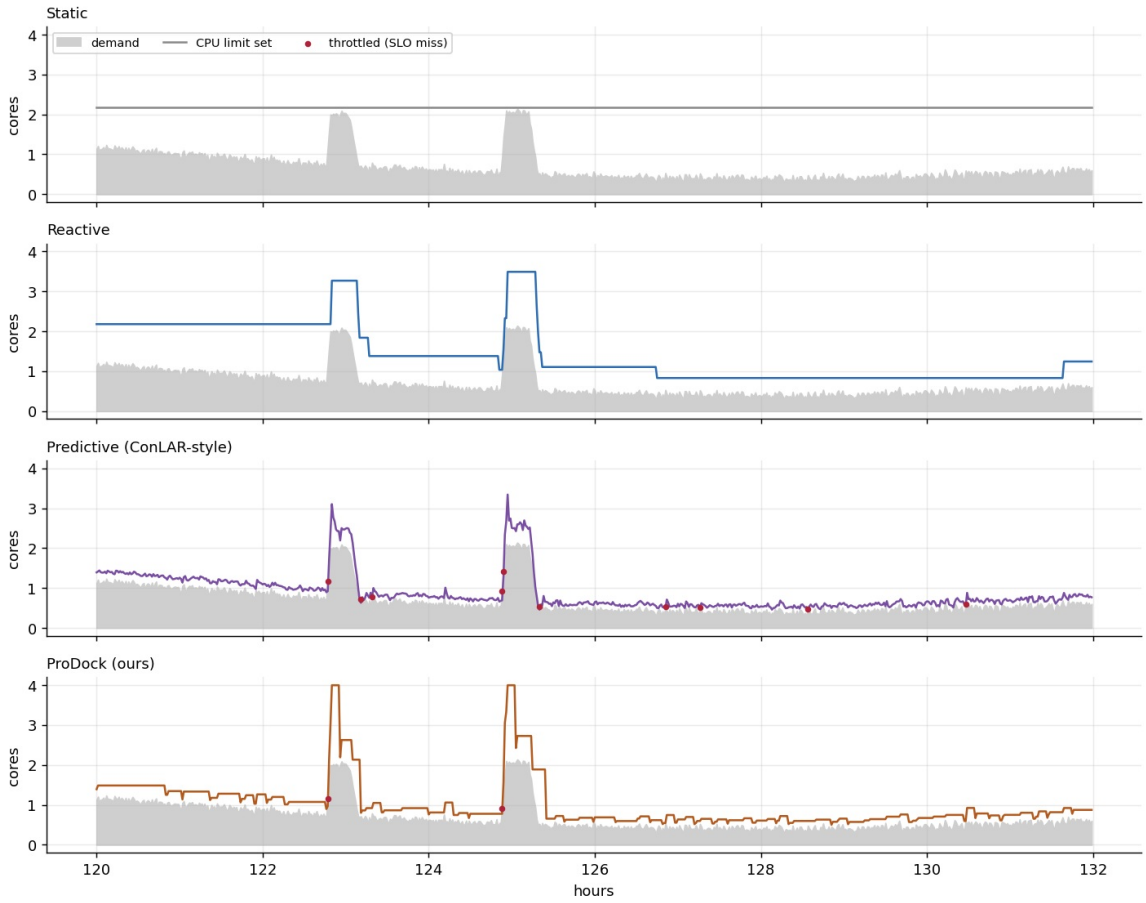
Model	Avg error (cores)	Error on peak minutes	How often it guessed too low
Naive (last value)	0.080	0.154	74%
Moving average	0.130	0.404	82%
Linear (Ridge)	0.059	0.098	42%
LSTM without burst score	0.053	0.105	54%
ProDock (Isolation Forest + LSTM)	0.061	0.102	30%

Be upfront about this one. Adding the burst score doesn't give the lowest average error. The plain LSTM is slightly better on that. What the burst score does is make the model guess too low much less often (30% vs 54%). For resource limits, guessing too low is the dangerous mistake because that's when containers get throttled. That's why ProDock wins on the policy table even though it doesn't win on raw error.

Graphs (in `figures/`)

- **Fig 1, workload traces.** The four containers over 7 days, coloured by train, validation and test. Shows how different the containers are.
- **Fig 2, burst detector.** The red dots are where the Isolation Forest flagged something unusual. Notice they land on the start of each spike, which is exactly when extra headroom helps most.
- **Fig 3, LSTM learning curve.** Training and validation error both fall. Validation doesn't shoot up, so the model isn't just memorising the training data.
- **Fig 4, forecast vs actual.** Five hours of unseen test data. The forecast tracks the real peak closely.
- **Fig 5, forecast error bars.** The table above as a chart.
- **Fig 6, policy timelines.** The best one to show. Same 12 hours of `web-frontend` under each policy. Static wastes a huge band of CPU. Reactive jumps late and stays too high. ConLAR-style hugs demand but gets caught at the start of spikes (red dots). ProDock jumps up the moment the burst score fires.
- **Fig 7, head-to-head bars.** The main results table as six charts.
- **Fig 8, trade-off curve.** We ran each method at many headroom settings. Bottom-left is better. ProDock's curve sits below the ConLAR-style curve everywhere, so at any level of waste, ProDock has fewer violations. This is the strongest single argument that the idea works, not just the tuning.
- **Fig 9, memory limits.** `batch-worker`'s memory jumps when its job starts. The zoomed panel on the right shows one job start: the x marks are OOM kills. Reactive climbs too slowly and gets killed for five minutes straight, ConLAR-style gets caught on the first minute, and ProDock stays above demand the whole time because its memory floor remembers the last job.

Fig 6. What each policy does to web-frontend over 12 test hours



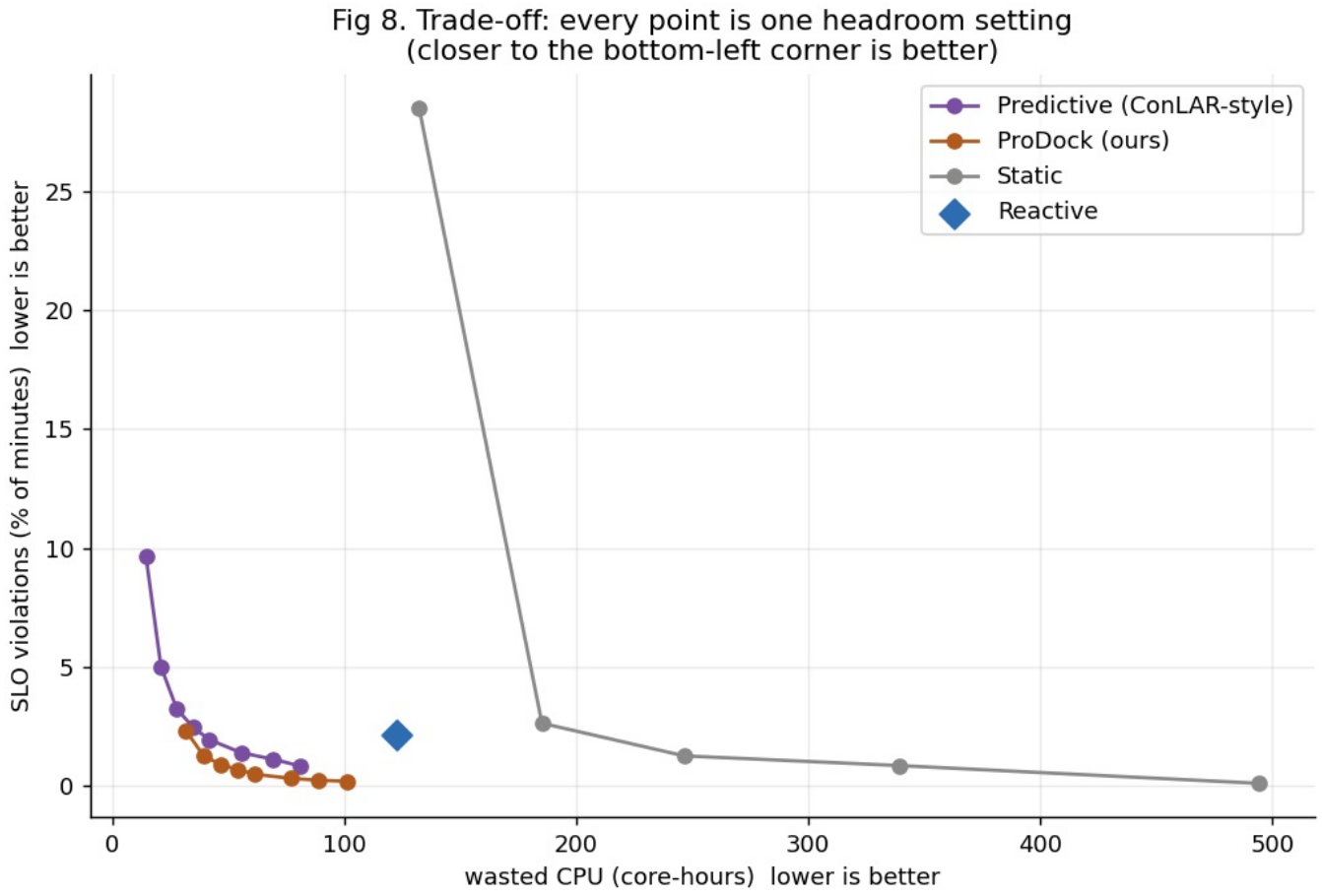
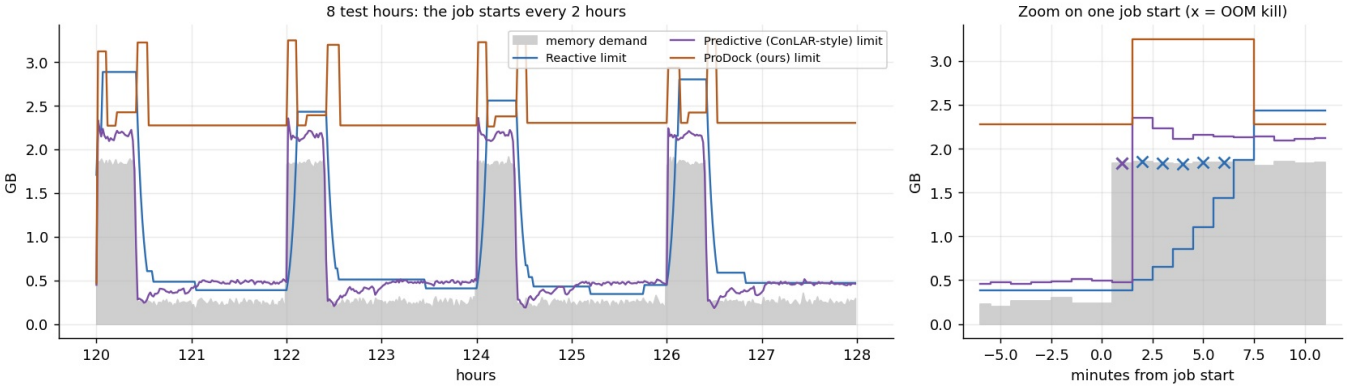


Fig 9. batch-worker memory limits. Demand above a policy's line means that policy's container is killed



Live demo

`python run_demo.py` replays test data and prints each decision. Around minute 7367 you'll see one throttled minute as a spike begins, then the burst score jumps, the allocator prints `burst-up` and raises the limit from 0.97 to 2.15 to 4.00 cores over the next three minutes. The whole 300-minute replay has 2 throttled minutes (0.7%). Good thing to run live in the review.

7. Limitations (say these before they're asked)

- **The data is simulated.** The results show the method behaves as designed. They don't prove production performance. Most papers in our own review have this same limitation (Section 5.3).
- **The model was trained on the simulated host.** `run_live.py` works with real Docker, but for real use you'd record a few days with `--record` and retrain. The real-Docker path wasn't tested in my environment because Docker wasn't available there, so test it on a laptop with Docker before showing it.
- **Scheduled jobs are hard to see coming.** `batch-worker`'s job runs every 2 hours, but the LSTM only looks back 30 minutes, so the first minute of each job is often missed. A "same time yesterday" feature would help.

- **The Isolation Forest scores on training data are in-sample.** The forest was fit on the same rows it scored during training. Test results are clean, but this is a small source of optimism.
- **ProDock uses more memory than the ConLAR-style predictor.** That's the price of avoiding OOM kills.
- **The LSTM learning curve is a bit bumpy.** It's a small model trained with plain NumPy, so that's expected. Early stopping keeps the best version.

8. How to run it

```
pip install -r requirements.txt
python tests/test_core.py
python run_experiment.py        # about 1 to 2 minutes, recreates results/ and figures/
python run_demo.py
python docs/draw_architecture.py
```

Everything uses a fixed random seed (7), so anyone who runs it gets the same numbers.

9. Questions you'll probably get

Why not just use Kubernetes? Lots of small businesses, edge devices and developer machines run plain Docker with no orchestrator. Our review found almost no AI allocation work aimed at them (Section 5.2).

Why an LSTM and not something newer? It's what our base paper uses, it's small enough to run next to the containers, and the review shows recurrent models are the standard for this job. A GRU or TCN would be reasonable swaps.

Why write the LSTM in NumPy? No heavy dependencies, runs on any laptop, and every step of the maths is visible. The gradient check test proves the backpropagation is right.

Why an Isolation Forest? It needs no labelled "burst" data, it's fast, and it's good at flagging rare, unusual points. That's exactly what the start of a spike looks like.

Why feed the burst score into the LSTM instead of just acting on it? A weird row doesn't always mean a spike is coming. Letting the LSTM learn from the score means it can tell the difference. We also use it directly in the allocator for extra headroom, as a safety net.

Why predict the peak and not the average? A limit has to cover the highest point in the next few minutes. If you plan for the average, you're under the limit about half the time.

Isn't resizing 1,966 times a lot? It's once every 6 minutes per container. `docker update` is just a cgroup write with no restart, so it's cheap. The ConLAR-style approach changed limits on almost every single minute.

Your forecast error isn't the lowest. Why is that okay? Because we care more about not guessing too low. ProDock under-predicts 30% of the time versus 54% for the plain LSTM, and that's what reduces throttling.

What's actually new here? The combination: a ConLAR-style Docker-level loop, plus an unsupervised burst score passed into the forecaster, plus burst-aware headroom, plus memory protection and anti-thrashing rules, all aimed at Docker without Kubernetes.

10. Suggested split for presenting

- **Person 1:** the problem, the gap from our review, and the base paper (Sections 1 and 2).
- **Person 2:** the architecture and how each layer works (Section 4, with the diagram).
- **Person 3:** experiment setup, results, graphs, the live demo, and limitations (Sections 5 to 7).

Everyone should be able to answer the questions in Section 9.